Tema 2.
Contendores

Objetivos

OBJETIVOS

- Comprender la diferencia entre contenedores y máquinas virtuales.
- Instalar y configurar contenedores.
- Configurar redes y persistencia de datos en contenedores.
- Desplegar aplicaciones en contenedores.
- Configurar y administrar entornos multicontenedor.

ÍNDICE

ÍNDICE

1. VIRTUALIZACIÓN Y CONTENEDORES
2. ¿QUÉ ES DOCKER?
3. INSTALACIÓN
4. COMANDOS BÁSICOS DE DOCKER
5. GESTIÓN DE IMÁGENES EN DOCKER
6. GESTIÓN DE ALMACENAMIENTO EN DOCKER
7. REDES EN DOCKER
8. CREACIÓN DE IMÁGENES CON DOCKER FILE
9. DOCKER COMPOSE

1. Virtualización y contenedores

1.1. Virtualización

- Wikipedia: “La virtualización utiliza el software para imitar las características del hardware y crear un sistema informático virtual.”
- Permite ejecutar más de un sistema virtual y múltiples sistemas operativos y aplicaciones en un solo servidor.
- Aumenta rendimiento del hardware incrementando el tiempo de procesamiento de un equipo.

1.1. Virtualización

Ventajas de la virtualización:

- Importante ahorro económico.
- Seguridad.
- Mayor aprovechamiento de recursos.
- Migración en vivo.
- Importante ahorro energético

1.1. Virtualización

Desventajas de la virtualización:

- Muchos sistemas dependen de un solo equipo físico.
- Penalizaciones en el rendimiento

1.1. Virtualización

Tipos de virtualización (I):

- Emulación: el sistema de virtualización suplanta vía software una arquitectura al completo (procesador, memoria, conjunto de instrucciones, ...) QEMU, Memu, Mame, Wine, ...

- Virtualización (por hw) de tipo 1 (hipervisor bare-metal): se instala directamente en el hw del servidor (sin s.o. anfitrión intermedio). Proporciona un rendimiento y seguridad superiores al acceder directamente a los recursos de hardware para las máquinas virtuales. VMware ESXi, Microsoft Hyper-V, Proxmox, KVM...

1.1. Virtualización

Tipos de virtualización (II):

- Virtualización (por hw) de tipo 2 (hipervisor hospedado): se instala un sw sobre un s.o. ya existente. Se accede a los recursos hw a través del s.o. del host anfitrión, por lo que el rendimiento es menor que el anterior VMware Workstation, VirtualBox, ...

- Virtualización ligera (basada en contenedores): sobre el núcleo del s.o. se ejecuta una capa de virtualización, permitiendo múltiples instancias aisladas de espacios de usuario. Cada una de estas instancias aisladas es lo que se llama contenedor.

https://danitic.wordpress.com/2020/10/08/2233/

1.2. Contenedores

- Un contenedor es un conjunto de procesos aislados, que se ejecutan en un servidor, y que acceden a un sistema de ficheros propio, tienen una configuración de red propia y acceden a los recursos del host (memoria y CPU).

- Utilizan para su funcionamiento el núcleo (kernel) del host donde se ejecutan (las máquinas virtuales cargan el s.o. completo).

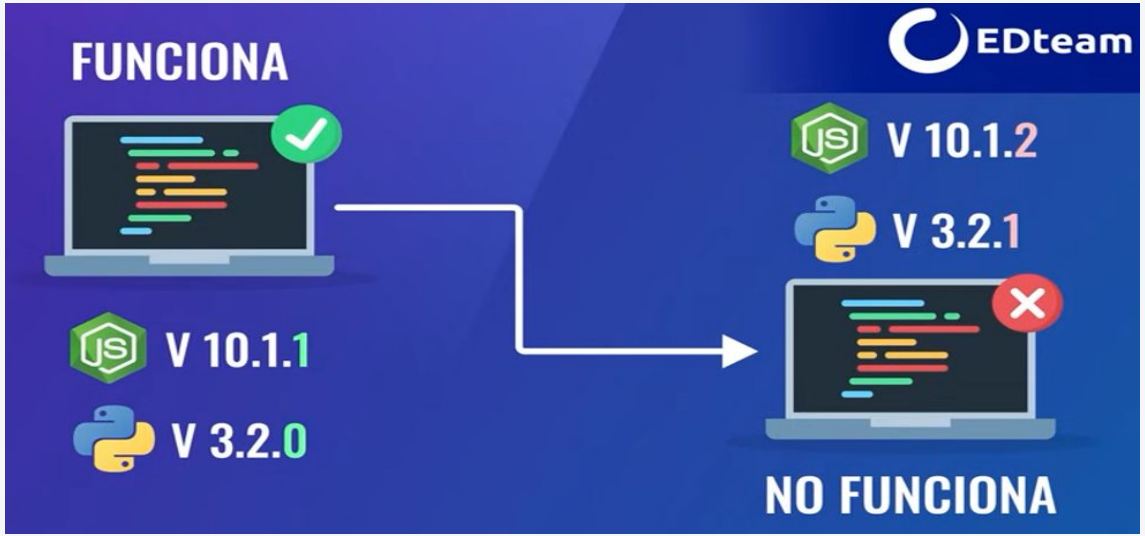
1.2. Contenedores

1.2. Contenedores

- El ciclo de vida del sw pasa por varias fases, varios ordenadores, varias capas el sw no está y no se ejecuta en un único entorno.

1.2. Contenedores

- Todas las computadoras del ciclo de vida tendrían que tener las mismas configuraciones, dependencias, etc.
- La realidad es otra: diferentes dependencias, diferentes configuraciones...



1.2. Contenedores

- ¿Y si metemos todo en una caja donde tenemos las configuraciones, dependencias, etc. para que la aplicación funcione?

1.2. Contenedores

Ventajas (I):

1. **Portabilidad**: se ejecutan de manera consistente en diferentes entornos, como desarrollo, pruebas y producción. Pueden moverse entre un portátil, un servidor, un cpd, a servicios en la nube sin hacer cambios.
2. **Aislamiento**: las aplicaciones se ejecuten sin interferencias con otras aplicaciones evitando conflictos de dependencias y compatibilidad.
3. **Eficiencia en el uso de recursos**: comparten el núcleo del s.o. y solo incluyen las bibliotecas y dependencias necesarias menos memoria, menos disco duro, menos recursos CPU.

1.2. Contenedores

Ventajas (II):

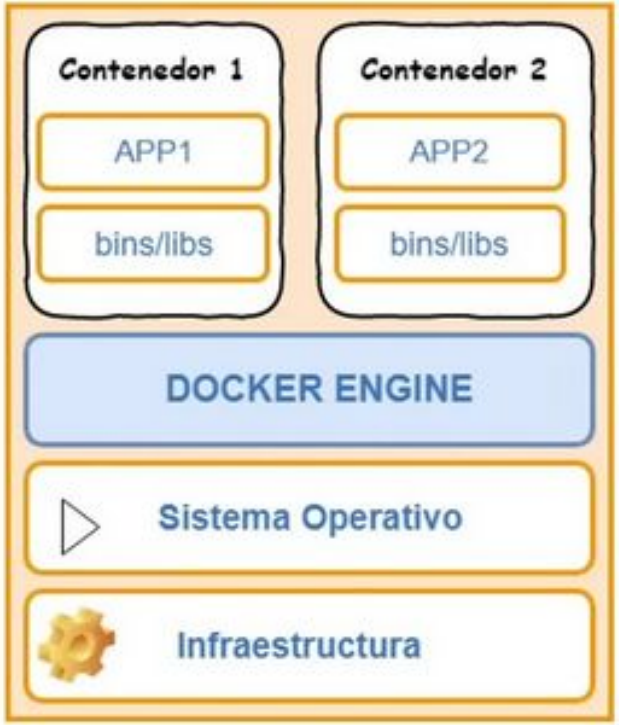
4. **Despliegue rápido**: permiten la creación, el despliegue y la escalabilidad rápida de aplicaciones CI/CD, recuperación entre fallos…

5. **Escalabilidad:** con kubernetes de puede escalar la aplicación automáticamente, aumentando/disminuyendo el número de contenedores a demanda

1.3. Máquinas virtuales vs Contenedores

Aspecto	Contenedores	Máquinas Virtuales (VM)
Arquitectura	Comparten el mismo núcleo del sistema operativo.	Cada VM incluye un sistema operativo completo y el hipervisor.
Peso	Son ligeros, ocupan pocos megabytes.	Son pesadas, pueden ocupar varios GB.
Arranque	Arrancan en pocos segundos.	El arranque puede llevar varios minutos.
Aislamiento	Aislamiento a nivel de proceso y sistema de archivos.	Aislamiento completo mediante el sistema operativo propio.
Rendimiento	Muy alto (no necesitan emular el hw).	Menor rendimiento por la sobrecarga del hipervisor.
Portabilidad	Se pueden ejecutar en cualquier entorno.	Menos portátiles, dependen del hipervisor y el sistema operativo.
Recursos	Uso eficiente, ya que comparten recursos del host.	Requieren recursos dedicados como CPU, RAM y almacenamiento.
Uso típico	Microservicios, despliegue rápido, entornos de desarrollo.	Aplicaciones monolíticas, sistemas operativos completos.

1.3. Máquinas virtuales vs Contenedores



1.4. Cuándo usar contenedores en producción

Entonces, si a priori todo son ventajas, ¿siempre habrá que usar contendores en aplicaciones en producción?

Dependerá de la naturaleza y arquitectura de la aplicación:

- Arquitectura monolítica
- Microservicios

1.4. Cuándo usar contenedores en producción

Arquitectura monolítica

- Todos los componentes de una aplicación (interfaz de usuario, lógica de negocio y la base de datos) se integran en una sola unidad unificada y autónoma.
- Facilita el desarrollo y la implementación para aplicaciones sencillas.

- Presenta desafíos de escalabilidad, mantenimiento y agilidad para aplicaciones complejas, ya que cualquier cambio requiere recompilar y rediseñar toda la aplicación.

1.4. Cuándo usar contenedores en producción

Arquitectura MONOLÍTICA

- La Aplicación está contenida en un solo modelado de clases
- Posee una sola lógica de negocio
- Posee una sola base de datos CENTRALIZADA para toda la aplicación
- Todo el código fuente de la aplicación se encuentra en un solo proyecto, que se compila en UN SOLO ARCHIVO EJECUTABLE.

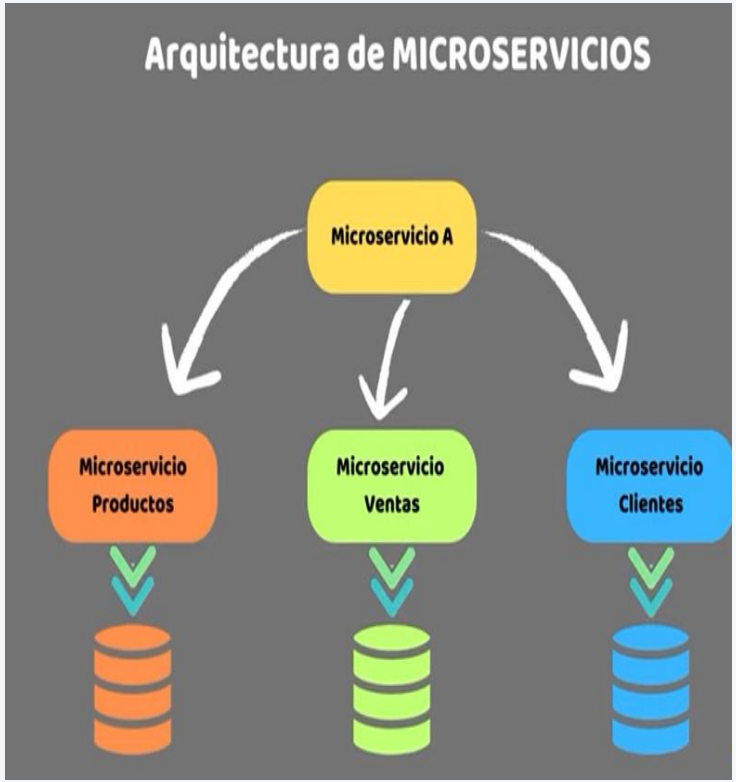
VENTAJAS y DESVENTAJAS de la Arquitectura MONOLÍTICA

- Fácil de desarrollar y mantener ✓
- No depender de servicios o aplicaciones externas ✓
- GRAN COMPLEJIDAD al crecer a gran escala ✗
- Mayor probabilidad de FALLOS y CUELLOS DE BOTELLA ✗

1.4. Cuándo usar contenedores en producción

Microservicios

- Se divide una aplicación grande en un conjunto de servicios más pequeños, independientes y autónomos.
- Cada microservicio una funcionalidad de negocio.
- Se comunica con otros servicios a través de APIs o mensajería.



1.4. Cuándo usar contenedores en producción

Microservicios

- Cada microservicio tiene su propia bbdd, su propio modelado y su propia lógica de negocio, usando apis para comuncarse entre sí.

- Cada microservicio es una miniaplicación; por ejemplo, una aplicación de cursos online tendrá un microservicio para los cursos, otro para facturación, otro para premium, otro para gestión de usuarios...

1.4. Cuándo usar contenedores en producción

- Cada microservicio estará en un contenedor y tendrán que comunicarse entre sí se irá complicando la cosa kubernetes

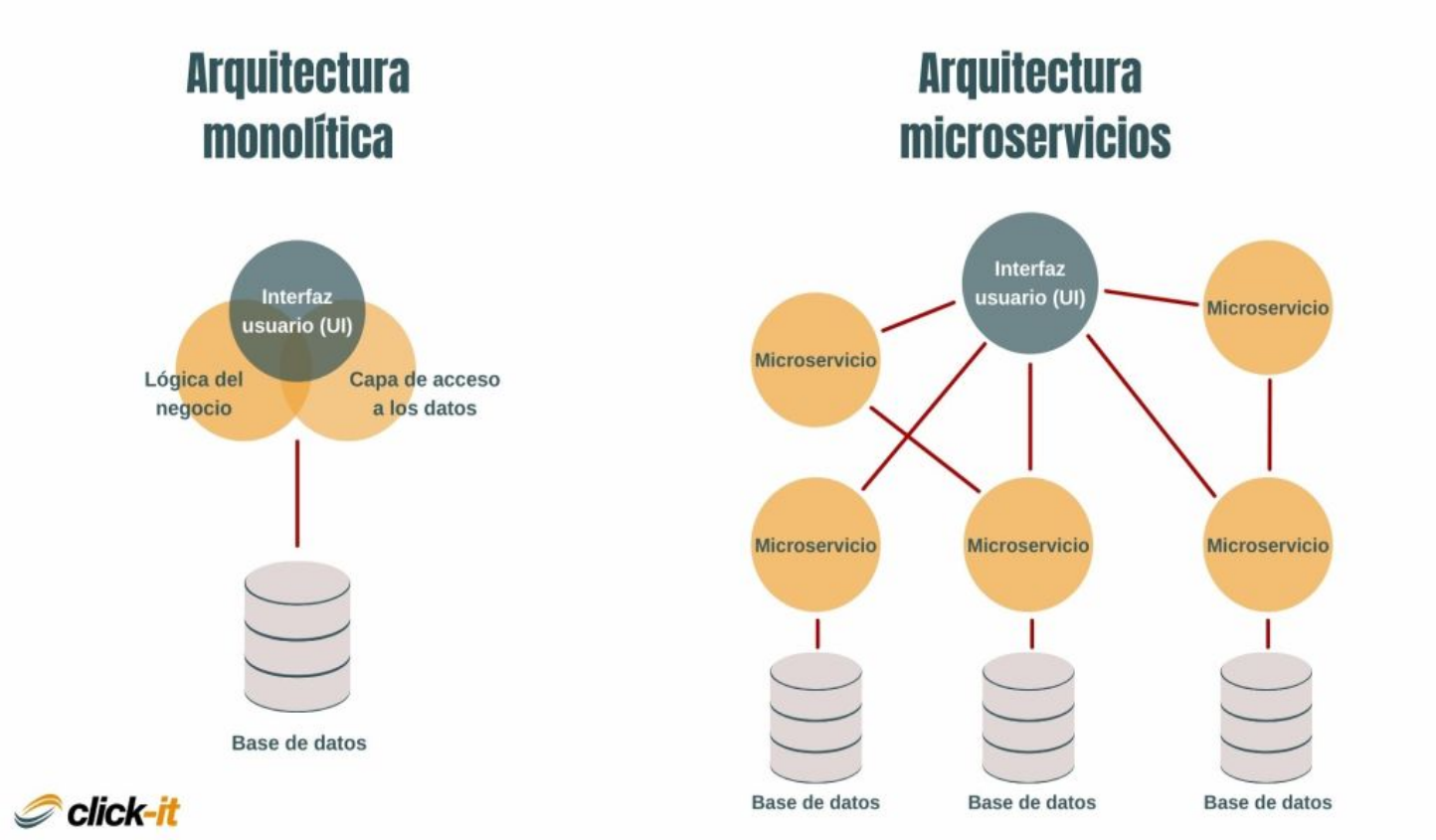
1.4. Cuándo usar contenedores en producción

<u>Ventajas</u>

- Escalabilidad
- Resiliencia
- Desarrollo mucho más rápido los servicios pueden ser desarrollados, desplegados y escalados independientemente.
- Si una parte deja de funcionar, el resto puede seguir funcionando

1.4. Cuándo usar contenedores en producción

1.4. Cuándo usar contenedores en producción



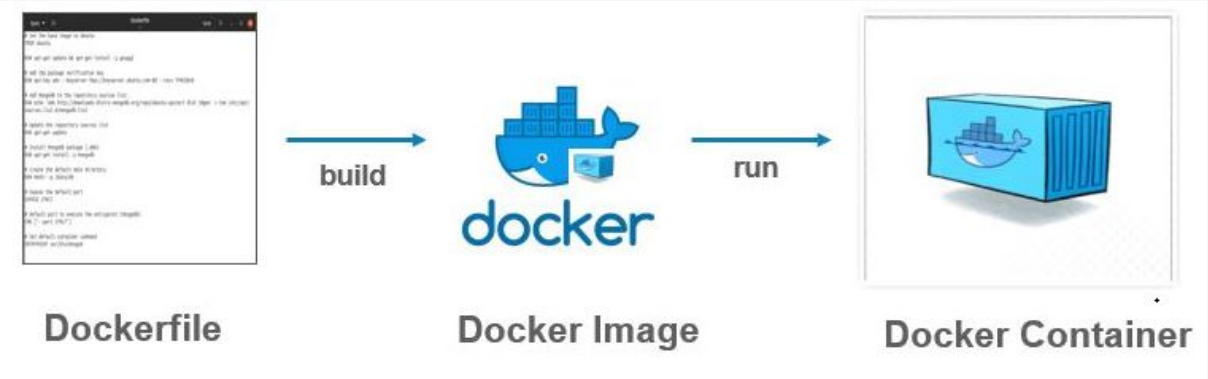
2. ¿Qué es Docker?

2. ¿Qué es Docker?

- Es una plataforma que facilita la creación, ejecución y gestión de contenedores.
- Creado en 2013 (en lenguaje Go) por la empresa dotCloud.
- Con Docker se puede:
  - Crear imágenes que contienen tu aplicación y sus dependencias.
  - Ejecutar esas imágenes como contenedores en cualquier máquina.
  - Administrar contenedores de forma sencilla con comandos intuitivos.

2. ¿Qué es Docker?

- Flujo de trabajo:



- Dockerfile se define el sistema base a usar, dependencias a instalar, archivos a usar, cómo ejecutar el programa...
- Image es el resultado de construir el dockerfile. Es inmutable
- Container instancia en ejecución de una imagen

3. Instalación

3.1. Instalación en Windows

1. Requisitos previos: tener la virtualización habilitada en la BIOS
2. Descargar Docker Desktop
3. Ejecutar el instalador y habilitar WSL 2 (subsistema de Windows para Linux) si fuera necesario.

   (No marcar la casilla “Allow Windows Container”
4. Verificación: desde Power Shell, ejecutar

   docker --version

3.2. Instalación en Linux (Ubuntu)

- Documentación oficial de Docker (https://docs.docker.com/engine/install/ubuntu/)
- Tres maneras de instalación de Docker Engine
- La óptima es a través de los repositorios con apt.

4. Comandos básicos

4.1. Ejecutar un contenedor

- Ejecutar un contenedor:

  docker run nombre_imagen (= docker container run nombre_imagen)

- ¿Qué ocurre al ejecutar un contenedor?

  1. El cliente docker indica al demonio Docker que debe crear un contenedor
  2. Si es la primera vez que se ejecuta el contenedor basado en una determinada imagen, se descargará del repositorio público Docker Hub y se guarda en nuestro registro local.
  3. Se crea el contenedor, el cual ejecutará una serie de acciones.
  4. Las acciones se envían al cliente Docker y se muestran en el terminal.
  5. El contenedor, una vez ejecutado el proceso, se para.

4.2. Imágenes vs Contenedores

- Una imagen es una plantilla de solo lectura. Es como un molde.
- Contienen:
  - Sistema de archivos
  - Código de la aplicación
  - Todas las dependencias necesarias

4.2. Imágenes vs Contenedores

- Un contenedor es una instancia de una imagen en ejecución.
- Al ejecutar docker run:
  - Coge una imagen
  - Crea una copia aislada (instancia) de ella
  - Ejecuta los procesos necesarios dentro de ese contendor
- Un contenedor puede:
  - Modificar su sistema de archivos temporalmente
  - Ejecutar procesos aisladamente
  - Tener su propio entorno de red y variables

4.3. Listar y eliminar contenedores

- Listar contenedores ejecutándose

  docker ps

- Listar todos los contenedores independientemente del estado:

  docker ps -a

```
isard@ubuntu22:~$ sudo docker ps -a
CONTAINER ID   IMAGE         COMMAND    CREATED          STATUS                     PORTS     NAMES
c3b955bcba1d   hello-world   "/hello"   27 minutes ago   Exited (0) 27 minutes ago            pensive_kare
```

- Eliminar contenedores:

  docker rm (id_contenedor | nombre_contenedor)

4.4. Listar y eliminar imágenes

- Listar imágenes

  docker images [nombre_imagen[:tag]]

- Eliminar imágenes:

  docker rmi nombre_imagen # elimina una imagen específica

  docker image prune –a # elimina cualquier imagen que no esté asociada con un contenedor en ejecución

  docker system prune –a #elimina todos los contenedores e imágenes parados

4.5. Nombrar / renombrar contenedores

- Asignar nombre desde el principio

  docker run --name nombre_contenedor nombre_imagen

  (docker run --name serv_web httpd:2.4)

- Renombrar un contenedor

  docker rename nombre_antiguo nombre_nuevo

4.6. Ejecutar contenedores interactivos

- Crear contenedor interactivo

  docker run –it - -name nombre_contenedor imagen bash

- Para volver a conectarnos al contenedor ya creado:

  docker start nombre_contenedor

  docker attach nombre_contenedor

- Si el contenedor ya está ejecutándose podemos ejecutar comandos directamente:

  docker exec -it nombre_contenedor bash ó

  docker exec nombre_contenedor bash -c comando

4.7. Configuración de variables de entorno

- A la hora de crear contenedores puede ser necesario crear variables de entorno para que el proceso que inicializa el contenedor pueda usarlas.

  Se usará –e (o --env):

docker run –it - -name nombre_contenedor -e nombre_variable=valor imagen bash

4.8. Mapeo de puertos

- Para acceder desde el host del contenedor al mismo, debemos mapear los puertos:

docker run –it - -name nombre_contenedor –p puerto_local:puerto_contendor imagen

4.9. Copiar archivos desde el anfitrión a un contendor

Podemos copiar archivos desde nuestra máquina a un contendor:

docker cp ruta_anfitrión nombre_contenedor:ruta_contendor

5. Gestión de imágenes

5.1. Búsqueda y descarga de imágenes

- Para buscar una determinada imagen tenemos dos opciones:
  - Desde Docker Hub
  - Con el comando *docker search XXX*

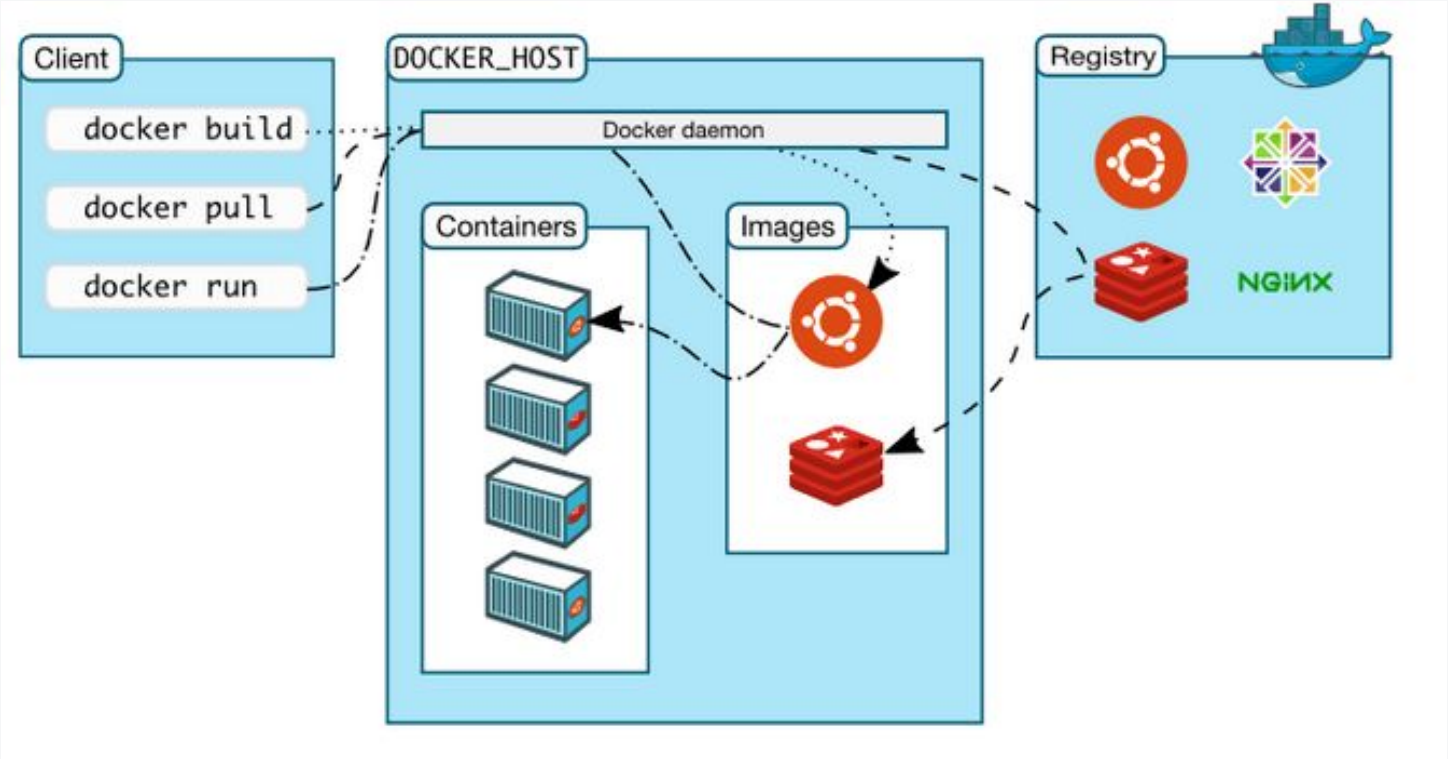
- Para descargar una imagen:

  docker pull nombre_imagen

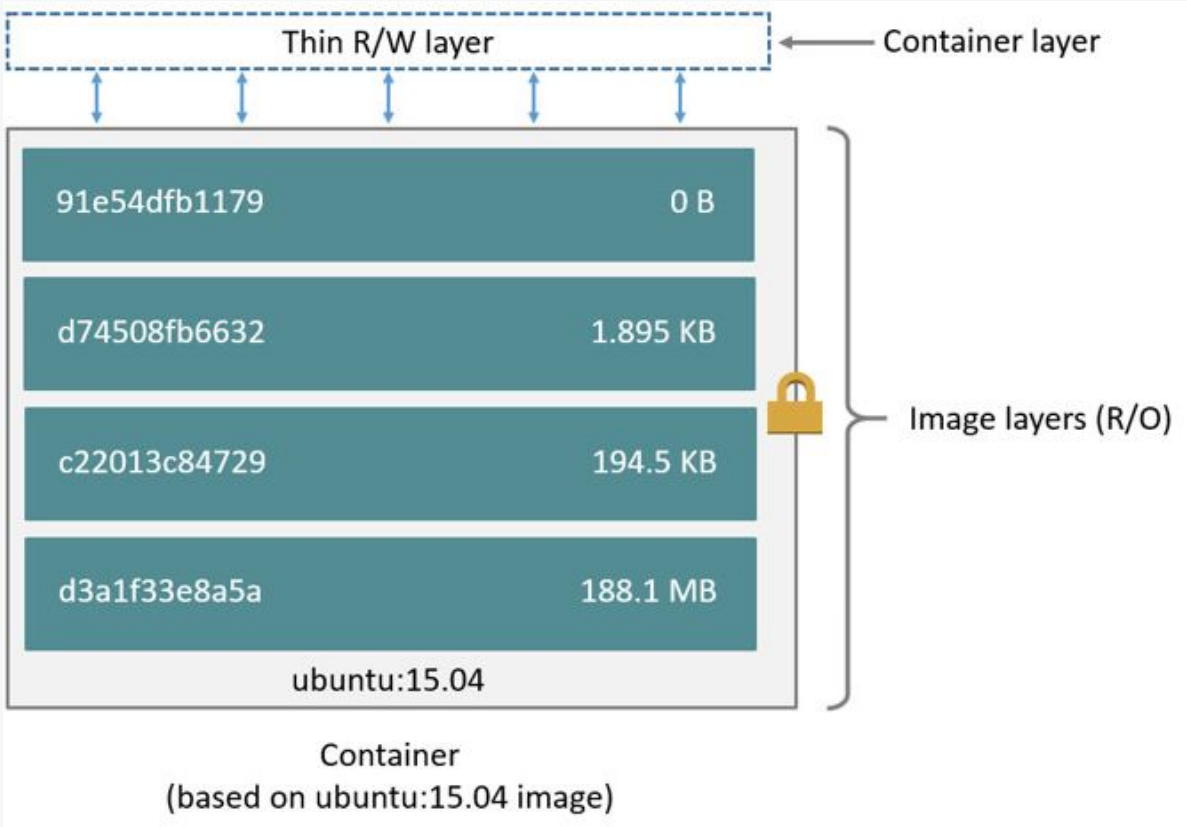
5.2. Registros de imágenes

- En el registro se almacena las imágenes generadas por el Docker Engine.
- Permite distribuir nuestras propias imágenes.
- Puede ser instalado gratuitamente en cualquier servidor (el proyecto nos ofrece Docker Hub).
- El nombre de una imagen tiene tres partes: usuario/nombre:etiqueta
  - Usuario que ha generado la imagen. Será nuestro nombre de usuario en Docker Hub (las imágenes oficiales no tienen nombre de usuario).
  - Nombre de la imagen.
  - Etiqueta: versiona las imágenes para control de cambios (por defecto latest).

5.2. Registros de imágenes



5.3. Organización de las imágenes



5.3. Organización de las imágenes

- Las imágenes se componen de una serie de capas, donde cada capa representa modificaciones o adiciones a la anterior.
- Las capas se apilan unas sobre otras (la capa superior es la más reciente), creando un sistema de archivos unificado.
- Las imágenes se construyen con una serie de instrucciones, creando cada una de ellas una nueva capa.
- Al ejecutar una imagen y crearse un contenedor, las capas se combinan para crear el sistema de archivos final.
- Al crear un contenedor ocupa muy poco espacio, ya que las capas de la imagen desde la que se ha creado se comparten con el contenedor

5.4. Creación de imágenes

- Además de usar imágenes descargadas de Docker Hub, podemos crear nuestras propias imágenes personalizadas dockerizar una aplicación.
- Hay dos maneras de hacerlo:
  - A partir de un contenedor
  - A partir de un Dockerfile (lo veremos en el punto 8 del tema)

5.4. Creación de imágenes

- A partir de un contenedor:
  1. Arrancamos un contenedor a partir de una imagen base
  2. Modificamos el contenedor
  3. Creamos una nueva imagen haciendo un commit creará una imagen con las capas de la imagen base, más la propia capa del contenedor
  4. Al ejecutar el contenedor hay que indicar el proceso que queremos que se ejecute por defecto

5.4. Creación de imágenes

- A partir de un contenedor:

1. Arrancamos un contenedor a partir de una imagen base

```
isard@ubuntu22:~$ docker run -it --name c_apache debian bash
```

2. Modificamos el contenedor

```
root@156d85aebe70:/# apt update && apt install apache2 -y
```

```
root@156d85aebe70:/# echo "<h1>Contenedor personalizado. Aniadido Apache a un debian</h1>" > /var/www/html/index.html
```

(...continúa...)

5.4. Creación de imágenes

3. Creamos una nueva imagen haciendo un commit □ creará una imagen con las capas de la imagen base más la propia capa del contenedor

```
isard@ubuntu22:~$ docker commit -a "Quqiue" -m "Añadir apache a imagen base debian" c_apache eruiz/mi_apache2:v1
sha256:bd570c453ef9f67762ee85cf16f2a21e89645b87014c1b7fb6220d9129f1c3c3
isard@ubuntu22:~$ docker images
                                                                                    i Info →  U  In Use
IMAGE                                    ID             DISK USAGE   CONTENT SIZE   EXTRA
alpine:latest                            4b7ce07002c6       12.8MB         3.8MB     U
debian:latest                            01a723bf5bfb        183MB        49.3MB     U
eruiz/mi_apache2:v1                      bd570c453ef9        359MB        96.9MB
```

4. Al ejecutar el contenedor hay que indicar el proceso que queremos que se ejecute por defecto

```
docker run -d -p 8282:80 --name servidor_web eruiz/mi_apache2:v1 bash -c "apache2ctl -D FOREGROUND"
```

5.5. Subir imágenes a DockerHub

1. Crearse una cuenta en DockerHub.

2. Etiquetar la imagen a subir

docker tag nombre-imagen-local usuario/nombre-imagen:etiqueta

3. Iniciar sesión en Docker Hub

docker login

4. Subir la imagen a Docker Hub

docker push usuario/nombre-imagen:etiqueta

5. (Descargar la imagen)

docker pull usuario/nombre-imagen:etiqueta

6. Almacenamiento en Docker

6.1. Persistencia de datos en Docker

- Los contenedores son efímeros ficheros, datos y configuraciones creados en los contenedores sobreviven a las paradas, pero no si el contenedor es borrado.

- Para almacenar de manera persistente:
  - Volúmenes
  - Bind mount

6.2. Volúmenes

Qué son los volúmenes

- Son directorios gestionados por Docker, almacenados en su propia ruta interna:
  - En Linux: /var/lib/docker/volumes/
  - En Windows (Docker Desktop): dentro de la VM de Docker

- Comando

  docker volume

6.2. Volúmenes

Características principales

- Docker controla totalmente el volumen.
- Se usan especialmente para bases de datos y datos críticos.
- No dependen de rutas del host → más portables.
- Se incluyen de forma nativa en Docker Compose.
- Mayor seguridad: el usuario del host no accede directamente a los datos (salvo que quiera).
- Un volumen puede ser montado en múltiples contenedores simultáneamente

6.2. Volúmenes

Ventajas

- Más seguro (aislado del host, evita borrados accidentales).
- Más portable (un volumen se replica igual en cualquier máquina).
- Mejor rendimiento en la mayoría de situaciones.
- Docker puede hacer backups, inspecciones y limpieza automática.

6.2. Volúmenes

<u>Cuándo usarlos</u>

- En producción
- Datos críticos
- Contenedores gestionados en Compose o Kubernetes / Swarm
- Cuando se necesita persistencia segura gestionada por Docker

6.3. Bind mounts

Qué son los bind mounts

- Consisten en montar una carpeta del host directamente dentro del contenedor

6.3. Bind mounts

Características principales

- Usan la ruta del host.
- Útiles para desarrollo: si editas un archivo en tu PC, se actualiza dentro del contenedor.
- Más flexibles, pero menos seguros

6.3. Bind mounts

Ventajas

- Perfectos para el desarrollo en tiempo real
- Permiten trabajar directamente con los archivos del host

Desventajas

- Menos portables: la ruta de tu equipo no tiene por qué existir en otro equipo
- Si se borra la carpeta del host se pierden los datos
- Problemas de permisos y sincronización

6.3. Bind mounts

<u>Cuándo usarlos</u>

- Proyectos donde editas archivos y ves cambios en caliente
- Compartir carpetas del host con el contenedor
- Cuando se necesita sincronización directa con capetas del host

6.4. Comparativa volúmenes - Bind mounts

Característica	Volumen Docker	Bind Mount
Quién lo gestiona	Docker	El usuario del host
Ubicación	Dentro de Docker (`/var/lib/docker/volumes`)	Ruta exacta del host
Seguridad	Alta (aislado)	Baja (depende del usuario)
Portabilidad	Muy alta	Muy baja
Rendimiento	Mejor	Puede ser más lento
Ideal para	BBDD, datos persistentes	Desarrollo, carpetas que editas tú
Riesgo de borrado accidental	Bajo	Alto

6.5. Configuración de volúmenes

- Comandos básicos

docker volumen create nombre_volumen #crea un volumen

docker volumen ls # lista los volúmenes creados

docker volumen inspect nombre_volumen # obtener info del volumen

docker volumen rm nombre_volumen # eliminar un volumen

6.5. Configuración de volúmenes

- Montar el volumen

docker run -d --name nombre_contenedor --mount type=volume,src=nombre_volumen,dst=ruta_en_el_contenedor -p puertos imagen

De otra manera (antigua) menos recomendada y que no podéis usar, pero por si lo veis, que lo conozcáis

docker run -d --name nombre_contenedor -v nombre_volume:ruta_en_el_contenedor -p puertos imagen

6.6. Configuración de bind mount

- Montar el bind mount

Primero, en el anfitrión se creará el directorio con los archivos que montaremos posteriormente.

El montaje será similar al comando con volúmenes:

docker run -d --name nombre_contenedor --mount type=bind,src=ruta_amfitrion,dst=ruta_en_el_contenedor -p puertos imagen

(Para inspeccionarlo *docker inspect nombre_contendor*)

6.7. Compartir un volumen entre varios contenedores

- Los volúmenes pueden ser compartidos por varios contendores…pero:
  - Para bases de datos no se debería hacer, pues se pueden corromper los datos
  - Para un servidor web estático no hay ningún problema (normalmente permisos de solo lectura: *--mount …,dst=/usr/local/apache2/htdocs:ro*
  - Para un servidor dinámico se puede compartir pero habría que evitar problemas de concurrencia

7. Redes en Docker

7.1 Redes en Docker

- Permiten que los contendores se conecten entre sí y con el exterior (internet) → al crear un contenedor, este se conecta a una red virtual (usando bridges e iptables) para asignarle una ip interna, pueda acceder al exterior, mapee puertos, ...

- Tipos de redes
  - Red host
  - Red bridge
  - Red none
  - Red overlan
  - Rev macvlan

7.1.1 Redes host

- Redes Host
  - El contenedor usa directamente la interfaz de red del host (no hay aislamiento de red)
  - No se le asigna dirección ip propia
  - Más rápido
  - Menos seguro
  - No hay puertos –p, ya que usa los del host (no requiere DNAT)
  - Cuando se necesita manejar un gran número de puertos
  - Para monitorización, casos donde el rendimiento es crítico

7.1.2 Redes Bridge

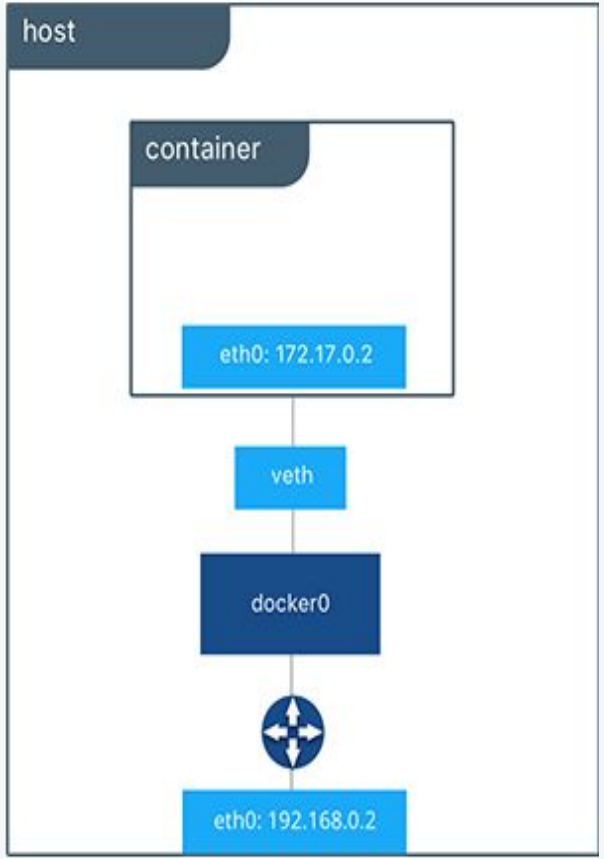
- Redes Bridge (son las que usa docker por defecto).
  - Las más usadas
  - Red privada interna del host
  - Contendores se comunican entre sí
  - El host puede acceder a los contenedores
  - Usa NAT para salir a internet
  - Pueden ser: por defecto o definidas por el usuario

7.1.2 Redes Bridge (por defecto)

a) Redes Bridge por defecto

- Red por defecto
- Direccionamiento 172.17.0.0/16.
- Problema: no hay resolución DNS por nombre de contenedor
- Usada para:
  - ✔ Aplicaciones normales
  - ✔ Web + base de datos
  - ✔ Microservicios en un solo host

(en producción serán bridges definidas)



7.1.2 Redes Bridge (por defecto)

(... contibnuación redes bridge por defecto...)

docker run -d -p 8080:80 --name contenedor1 alpine

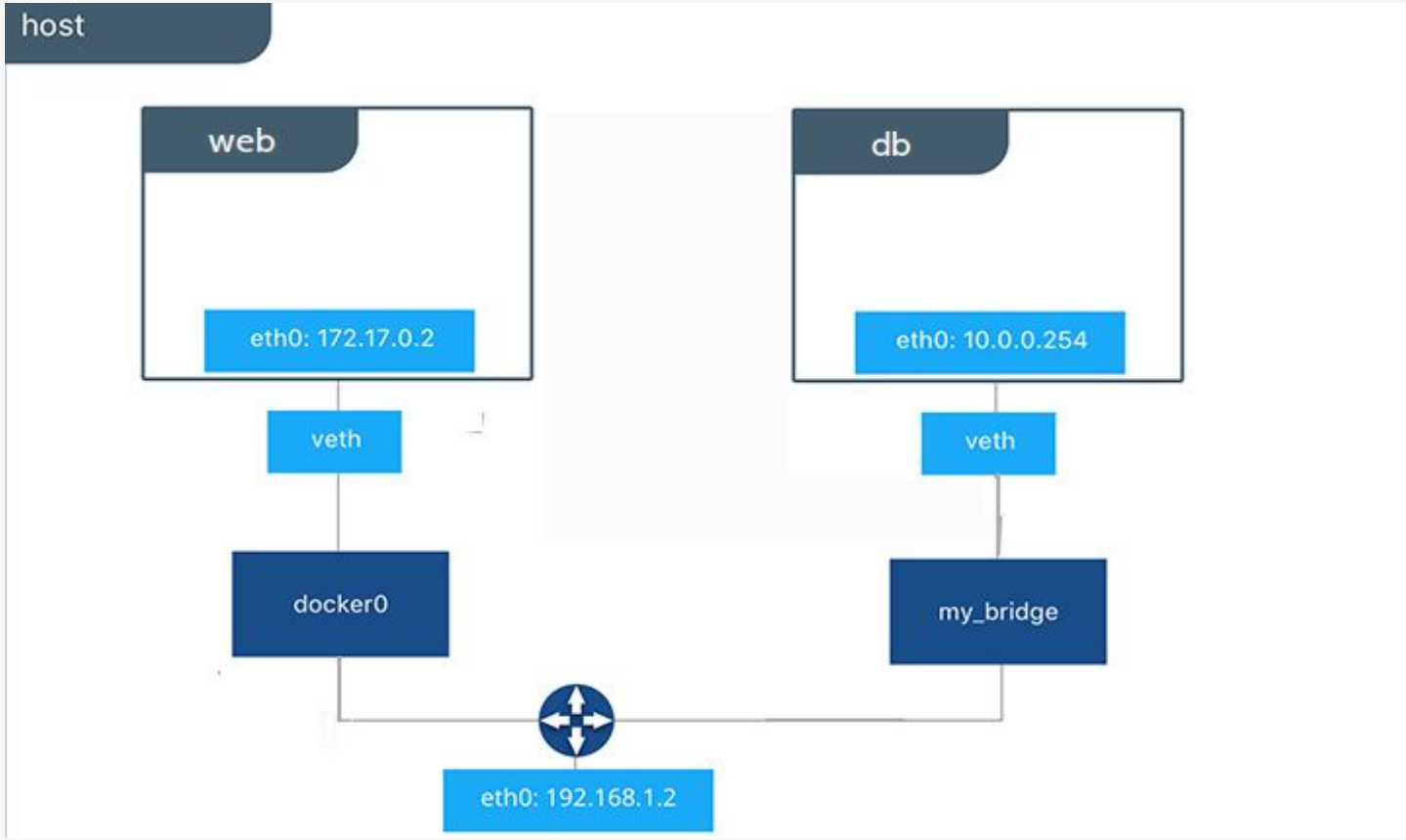
Son las que hemos usado hasta ahora

7.1.2 Redes Bridge (definidas por el usuario)

b) Redes Bridge definidas por el usuario

- Permiten interconectar los contenedores que se encuentran en el mismo segmento de red (y por lo tanto aislar contenedores en distintas redes) y dentro del mismo host. Desde cada una de las redes solo se podrá acceder a los equipos de esa misma red.
- Se puede conectar en caliente un contenedor a una red bridge definida por el usuario.
- Permiten gestionar de manera más segura el aislamiento de contenedores
- Proporcionan DNS, permitiendo el acceso entre contenedores usando su nombre.
- Usada en producción (no usar bridge por defecto en producción)

7.1.2 Redes Bridge (definidas por el usuario)



7.1.3 Redes none

- Redes None
  - El contenedor no tiene red, ni externa ni con otros contenedores (totalmente aislado)
  - Usada en contenedores que no necesitan red
  - No tiene configurada ninguna ip (tiene la dirección de loopback)
  - En procesos batch, scripts, seguridad extrema

7.1.4 Otros tipos de redes

- Overlay comunica contenedores de distintos hosts.
  - Usada Docker Swarm (clústeres)
  - En microservicios distribuidos
  - Alta disponibilidad

- Macvlan contenedor obtiene ip de la red física (es una máquina más de la LAN)
  - Usada en sistemas que requieren ip fija
  - En integración con infraestructura antigua

- Ipvlan permite direccionamiento ipv4 e ipv6 y recrear vlans

7.1.5 Comparativa redes

Red	Aislamiento	DNS	Multi-host	Uso típico
bridge	Medio	❌ / ✅	❌	Apps normales
host	❌	—	❌	Alto rendimiento
none	✅✅✅	—	❌	Sin red
overlay	✅✅	✅	✅	Swarm
macvlan	Bajo	❌	❌	IP física

7.1.6 Tabla de equivalencia con redes virtual box

Docker	VirtualBox equivalente	Motivo
`bridge` por defecto	**NAT Network**	Red compartida + NAT
`bridge` personalizada	**NAT Network (configurada)**	Igual pero mejor
`host`	Adaptador puente	Sin NAT
`none`	Sin adaptador	Aislamiento total
`overlay`	No directo (↔ VXLAN)	Red entre hosts
`macvlan`	Adaptador puente + IP propia	IP real en LAN

7.2 Comandos generales

Comando	Descripción
docker network ls	Lista todas las redes disponibles.
docker network create mynetwork	Crea una nueva red llamada "mynetwork".
docker network inspect mynetwork	Muestra detalles de la red "mynetwork".
docker network connect mynetwork container	Conecta un contenedor a una red existente.
docker network disconnect mynetwork container	Desconecta un contenedor de una red.
docker run --network=mynetwork	Ejecuta un contenedor en una red específica.

7.3 Configuración redes host

- Creamos un contenedor web que espera peticiones por le puerto tcp/80

  docker run --rm -d --name c_nginx --network host nginx

- Accedemos desde el navegador con *localhost*

7.4.1 Configuración redes bridge por defecto

- Son con las que hemos trabajado hasta ahora al crear contenedores a los que accedíamos

docker run -d --name contenedor1 -p 8080:80 nginx

7.4.2 Configuración redes bridge definidas por usuario

- Recordatorio: proporcionan DNS. Usadas en entornos de producción

<u>Pasos</u>

1. Creación de la red con direccionamiento por defecto

docker network create –d bridge --subnet ip_red/máscara red1

(si se omite *--subnet ip_red/máscara,* asignará docker una por defecto)

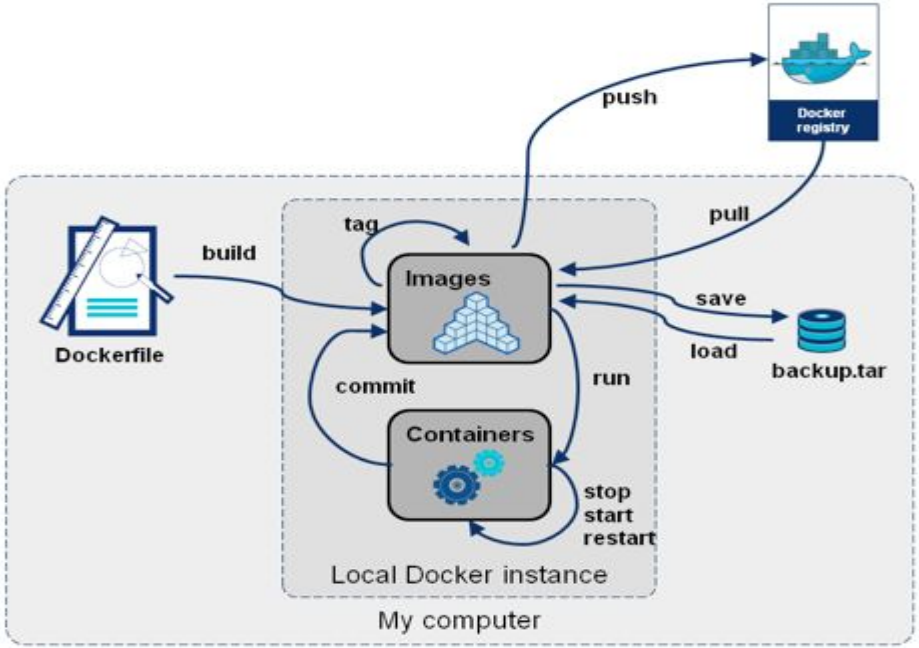
2. Creación del contenedor para que use la red creada

docker run -d --name servidorweb --network red1 nginx

8. Creación de imágenes con Dockerfile

8. Creación de imágenes con Dockerfile

- En el punto 5 vimos como crear una imagen a raíz de un contenedor. Vamos a estudiar ahora otra manera de hacerlo, a través de un fichero llamado dockerfile, que ofrece varias ventajas.



8.1 Ventajas de usar un Dockerfile

Ventajas con respecto al método visto anteriormente:

- Se puede reproducir la imagen fácilmente la configuración está en un fichero, no hay que recordar los comandos usados antes del commit para reproducirla de nuevo. Podemos subir el Dockerfile a git, teniendo control sobre los cambios.

- Podemos indicar el proceso que se ejecutará por defecto en el contenedor creado desde la imagen, sin que sea el configurado por defecto en la imagen base.

- Si hay problemas con la imagen base (actualizaciones, fallo de seguridad...), modificando una línea descargamos la nueva versión de la imagen sin afectar al resto de las capas y configuraciones.

8.2 El fichero Dockerfile

- Un fichero **Dockerfile** es un conjunto de instrucciones que serán ejecutadas **de forma secuencial** para construir una nueva imagen docker.

- Cada una de estas instrucciones crea una nueva capa en la imagen que estamos creando.

- Estructura fundamental:
  - Indicamos imagen base: FROM
  - Metadatos: LABEL
  - Instrucciones de construcción: RUN, COPY, ADD, WORKDIR
  - Configuración: usuarios, puertos, variables de entornos: USER, EXPOSE, ENV
  - Instrucciones de arranque: CMD, ENTRYPOINT

8.2 El fichero Dockerfile

- Instrucciones de un Dockerfile https://docs.docker.com/reference/dockerfile

FROM php:7.4-apache #especifica la imagen base

LABEL version="1.0" #añade metadatos a la imagen mediante clave=valor

WORKDIR /usr/local/apache/htdocs #establece directorio de trabajo (= cd ...)

COPY --chown=www-data:www-data webs /var/www/html #copia ficheros

ADD https://example.com/static.tar.gz /var/www/html/ #copia "especial"

RUN apt update && apt install -y nano #ejecuta una orden creando una nueva capa

ENV WEB_DOCUMENT_ROOT=/var/www/html #establece variables de entorno

EXPOSE 80 #informa de los puertos expuestos. **Es meramente informativo**

ENTRYPOINT ["echo"] #establece el ejecutable que se lanza al crear el contenedor

CMD ["hola"] #parámetros para el ENTRYPOINT

8.2 El fichero Dockerfile

- Consideraciones (I)
  - *COPY --chown=www-data:www-data webs /var/www/html* → copy usuario(opcional) origen(carpeta anfitrión) destino(directorio contenedor)
  - *ADD* → como copy, pero para copiar páginas, archivos comprimidos que se descomprimen automáticamente en el contenedor. Hay que usar COPY excepto en los casos “especiales” comentados
  - *RUN ... -y nano* → muy importante –y, pues no hay interacción con el usuario
  - *ENV* → no se puede usar con ENTRYPOINT Y CMD

8.2 El fichero Dockerfile

- Consideraciones (II). ENTRYPOINT y CMD

  - CMD se sobreescribe al hacer Docker run

    docker run mi_imagen bash → cmd queda reemplazado por bash

  - Sólo el último CMD cuenta

    ENTRYPOINT ["echo"]

    CMD ["hola"]

    CMD ["adios"] → mostrará adios

8.2 El fichero Dockerfile

- Consideraciones (II). ENTRYPOINT y CMD
  - Tanto con uno como con el otro, se puede especificar la aplicación a ejecutar y luego el argumento:

CMD ["python", "app.py"] *ENTRYPOINT ["python", "app.py"]*

Pero es mejor separar el comando y los argumentos si es una imagen específica y no genérica:

Forma	¿Correcta?	¿Flexible?
ENTRYPOINT ["python","app.py"]	✅	❌
ENTRYPOINT ["python"] + CMD ["app.py"]	✅✅	✅
CMD ["python","app.py"]	✅	⚠️ Se sobrescribe

8.2 El fichero Dockerfile

- Consideraciones (II). ENTRYPOINT y CMD
  - Separar entrypoint y cmd da flexibilidad:

    ENTRYPOINT ["python"]

  CMD ["app.py"]

Si creamos la imagen y ejecutamos un contenedor de ella (*docker run mi-contenedor-python*), ejecutará la aplicación app.py en Python, pero si al crear el contenedor hacemos:

docker run mi-contenedor-python app2.py, sobreescribirá cmd ejecutando app2.py

8.3 Creación de imágenes a partir del Dockerfile

Pasos

1. Crear el contexto (directorio donde meteremos el dockerfile y los archivos necesarios)

2. Crear la imagen con

   docker build –t nombre_imagen directorio_contexto

   docker build –t ddeccn/ubuntunano:v1 .

8.4 Buenas prácticas al crear el Dockerfile

- Usar imágenes fiables de fuentes oficiales (Imágenes oficiales de Docker, Imágenes oficiales de Docker para .NET).

- Evitar la etiqueta latest para las imágenes base.

- Usar varias LABEL en una sola línea en vez de carias etiquetas LABEL:

  *LABEL version="0.0.1" *

  fecha="2022-03-03"

- Usar ADD solo cuando sea necesario.

- Al usar el comando COPY, copiar solo lo necesario:

  ~~*COPY ./app*~~ → *COPY server.js /app*

8.4 Buenas prácticas al crear el Dockerfile

- No instalar paquetes innecesarios y elimina la caché que usa en la instalación (en distribuciones Debian/Ubuntu, apt lo hace automáticamente)

- Minimizar el número de capas: debemos encontrar el balance entre la legibilidad del fichero Dockerfile y minimizar el número de capas que generamos.

- En relación con lo anterior, agrupar los comandos relacionados en un solo RUN:

```
RUN apt update \
&& apt install -y curl vim \
&& rm -rf /var/lib/apt/lists/*
```

¡OJO! Borrar en otro RUN no reduce el tamaño, ya que el borrado sería en otra capa

8.4 Buenas prácticas al crear el Dockerfile

- Usar un archivo .dockerignore donde se indicarán los archivos o directorios que se deben ignorar al construir la imagen (resultados de compilación, carpetas de git carpetas relativas al entorno de desarrollo...).

- Usar la forma Exec (ENTRYPOINT ["comando", "param1"]) en lugar de la forma shell (ENTRYPOINT comando param1).

- El Entrypoint debe ser claro y directo. Si se necesita una lógica compleja, es mejor encapsularla en un script y llamar a ese script desde Entrypoint.

- Los contenedores creados a partir de la imagen que vamos a generar con nuestro Dockerfile deben tener una mínima configuración.

8.5 Distribución de imágenes

Dos opciones

Distribución a partir de un fichero

1. Guardar la imagen en un .tar

   docker save usuario/nombre_imagen:etiqueta > nombre_fichero.tar

   docker save ddeccn/ubuntunano:v1 > ubuntunano.tar

2. Distribuir el fichero .tar

3. Añadir el fichero al repositorio local

   docker load –i nombre_fichero.tar

8.5 Distribución de imágenes

Distribución usando Docker hub

1. Accedmos a Docker Hub con el comando Docker login

   docker login

2. Subimos la imagen

   docker push usuario/nombre_imagen:etiqueta

3. Para descargar la imagen

   docker pull usuario/nombre_imagen:etiqueta

8.6. Ciclo de vida de aplicaciones Docker

1. Desarrollo de la aplicación
2. Creación de la imagen Docker
3. Probar la aplicación en entorno de prueba (desarrollo)
4. Distribución de la imagen
5. Implantación de la aplicación en producción
6. Modificación de la aplicación
7. Vuelta al paso 2

8.7 Eliminar objetos Docker no usados

- Eliminar imágenes huérfanas (colgadas o dangling. Son imágenes intermedias generados y que no forman parte de ninguna imagen) *docker image prune*
- Eliminar imágenes sin contendores *docker image prune –a*
- Eliminar contendores parados *docker container prune*
- Elimina redes que no se están usando *docker network prune*
- Elimina volúmnes no usados *docker volume prune*
- Elimina todos los elementos no usados *docker system prune*
- Elimina todos los elementos no usados

  incluidos volúmenes *docker system prune --volumes*

9. DOCKER COMPOSE

9.1. Orquestación de contenedores

- La orquestación de contenedores es el proceso de gestionar automáticamente el despliegue, la administración, la escalabilidad y la comunicación entre múltiples contenedores que forman parte de una aplicación distribuida.

- En entornos reales, las aplicaciones están compuestas por varios servicios que necesitan funcionar coordinadamente. Por ejemplo:
  - Un servidor web (Nginx o Apache).
  - Una base de datos (MySQL, PostgreSQL).
  - Un backend (Node.js, Python, Java).

- Cada uno puede estar en un contenedor diferente administrándose mediante orquestación.

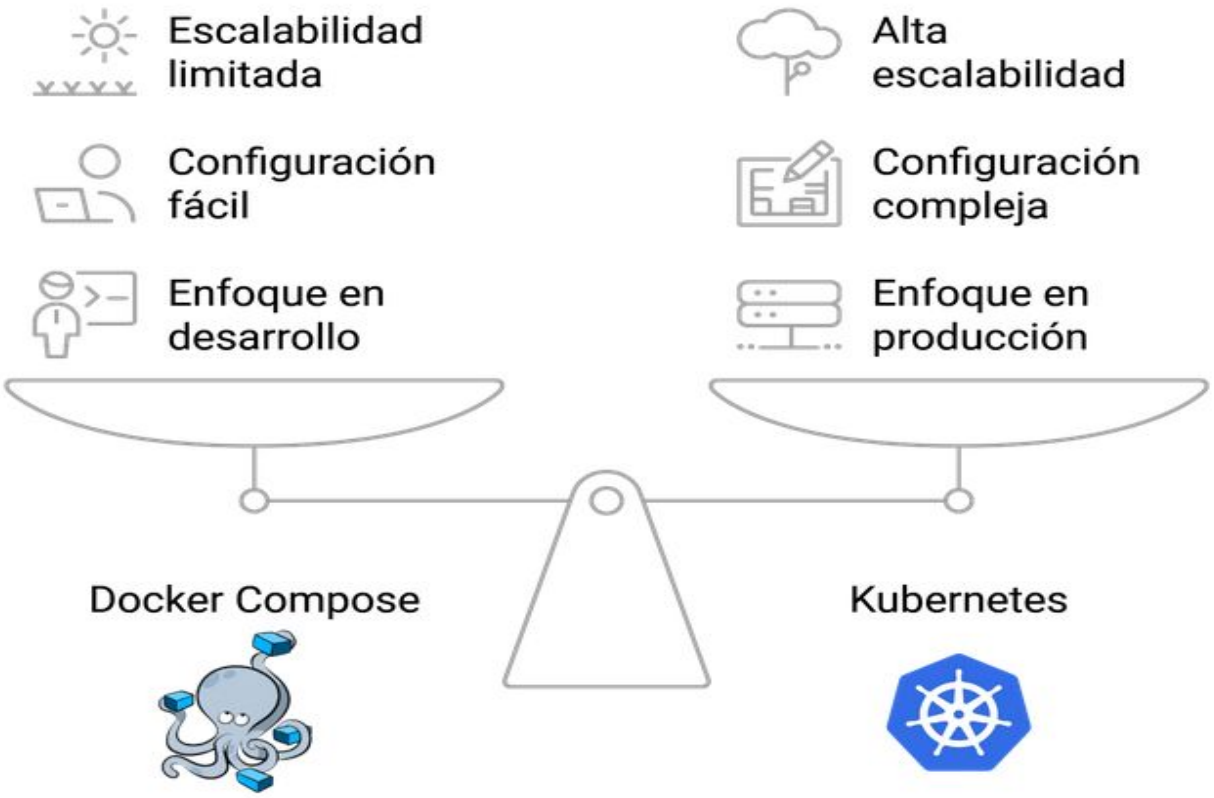
- Ejemplos de orquestadores: Docker compose, Docker Swarm, Kubernetes

9.1. Orquestación de contenedores

- La orquestación permite:
  - Levantar todos los contenedores con un solo comando.
  - Configurar redes internas para que los contenedores se comuniquen.
  - Gestionar volúmenes compartidos de datos.
  - Escalar los servicios (aumentar o reducir réplicas).
  - Supervisar y reiniciar contenedores en caso de fallo.

- Ventajas de la orquestación
  - Automatiza tareas complejas que serían muy tediosas si se gestionaran manualmente.
  - Mejora la fiabilidad y disponibilidad de las aplicaciones.
  - Facilita la escala horizontal de servicios.
  - Hace que la infraestructura sea portátil y reproducible.
  - Permite tener un entorno de desarrollo y producción consistente.

9.1. Orquestación de contenedores



9.2. Docker Compose

- Hemos visto que cada uno de los servicios de las aplicaciones se ejecutarán en un contendor distinto es necesario sincronizarlos, orquestarlos, haciendo que los contenedores trabajen en un mismo escenario.

- Docker Compose permite definir y correr aplicaciones que necesitan utilizar diferentes contenedores, automatizando la creación de esta con una única ejecución, sin necesidad de ejecutar múltiples docker run.

- El escenario se define en un fichero llamado compose.yaml (o compose.yml o docker-compose.yaml o docker-compose.yml por compatibilidad con versiones anteriores).

9.2. Docker Compose

- Ventajas de usar Docker compose
  - Hacer todo de manera declarativa sin necesidad de repetir todo el proceso cada vez que se construye el escenario.
  - Mantener la configuración centralizada y versionada fácil distribución del archivo Docker Compose, aumentando colaboración entre equipos.
  - Levantar los contenedores que necesita la aplicación de una sola vez.
  - Garantizar que los contenedores se arrancan en el orden adecuado (por ejemplo, lo primero el servidor de bases de datos).
  - Asegurarnos de que hay comunicación entre los contenedores.
  - Portabilidad entre entornos: permite el uso de variables para personalizar su composición para diferentes entornos o usuarios.

9.3. Instalación de Docker compose

- https://docs.docker.com/compose/install

(no hace falta instalar nada, al instar Docker, se instala)

9.4.1 El archivo Docker compose (compose.yaml)

- Estructura general

```yaml
services:
  service1:
    # Configuración del servicio 1

  service2:
    # Configuración del servicio 2

networks:
  network1:
    # Configuración de la red 1

volumes:
  volume1:
    # Configuración del volumen 1
```

9.4.1 El archivo Docker compose (compose.yaml)

- https://docs.docker.com/reference/compose-file/

```yaml
services:        # Sección principal: define los contenedores
  web:           # Nombre del servicio
    image: nginx:latest    # Imagen de Docker a usar
    ports:
      - "8080:80"          # Mapeo de puertos: host:contenedor
    volumes:
      - ./html:/usr/share/nginx/html  # Mapeo de volúmenes
    depends_on:
      - app                # Este servicio depende de "app"

  app:
    build: ./app           # Construye desde el Dockerfile ubicado en ./app
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
```

9.4.1 El archivo Docker compose (compose.yaml)

- Principales definiciones

version # desde la versión 2 de compose ya no se usa, deprecado

services # cada entrada bajo services define un contenedor (un servicio)

image # imagen a usar, puede der de Docker Hub o propia

build # construye la imagen que usa a partir de un docker file

ports # define el mapeo de puertos puerto_host:puerto_contenedor

(...)

9.4.1 El archivo Docker compose (compose.yaml)

environment # define variables de entorno dentro del contenedor

depends_on #especifica el orden de arranque de los servicios (cuando se hará docker start...aunque esto no garantiza que se creen en ese orden)

entrypoint # igual que en dockerfile

command # igual que el cmd en dockerfile

volumes # monta archivos o directorios locales dentro del contenedor

networks # define el tipo de red y su configuración

9.4.2 El archivo Docker compose. image / build

- Image -> cuando usamos una imagen ya construida

Se especifica la imagen a usar, bien de un repositorio (Dockerhub) o local

```
services:
  web:
    image: alpine:latest
```

- Build -> cuando se construye una imagen con Dockerfile en el momento

```
services:
  app:
    build: . # ruta relativa al dockerfile desde donde está el compose.yaml
    image: nombre_imagen  # asigna un nombre para reutilizar en otros
                                                  servicios
```

9.4.3 El archivo Docker compose. Volumes

- Volúmenes nombrados

```
services:
  db:
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

- Volúmenes tipo bind mount

```
services:
  app:
    volumes:
      - ./src:/app/src   #ruta
```

9.4.4 El archivo Docker compose. Networks

```
services:
  servicio1:
    networks:
      - red_a
servicio2:
  networks:
      - red_a
      - red_b
  servicio3:
    networks:
      - red_b
networks:
  red_a:
    driver: bridge
  red_b:
    driver: bridge
```

```
servicio1:
  networks:
    red_a:
      ipv4_address: 172.20.0.10 # asigna esta ip al contenedor (legacy)
servicio2:
  networks:
    red_a:
    red_b:
      ipv4_address: 172.21.0.20
servicio3:
  networks:
    red_b:
networks:
  red_a:
    driver: bridge
    ipam:
      config:
        - subnet: 172.20.0.0/24
          ip_range: 172.20.0.128/25
          gateway: 172.20.0.1
  red_b:
  ...
```

9.5. Comando Docker compose

docker compose up -d # Ejecuta los contenedores de forma desatendida

docker compose up -d --scale nombre_servicio=4 # crea 4 contenedores iguales

docker compose stop # Detiene los contenedores lanzados con docker compose up

docker compose run # Inicia los contenedores que estén parados

docker compose rm # Borra los contenedores parados del escenario (con las opción -s elimina también los contenedores en ejecución)

docker compose pause/unpause servicio # Pausa/reanuda los contenedores lanzados

9.5. Comando Docker compose

docker compose restart # Reinicia los contenedores (ideal para reiniciar servicios con nuevas configuraciones)

docker compose logs # Muestra los logs de todos los servicios del escenario

docker compose logs servicio1: # Muestra los logs del servicio llamado servicio1 que estaba descrito en el compose.yaml.

docker compose exec servicio1 /bin/bash # Ejecuta una orden, en este caso /bin/bash en el servicio servicio1

docker compose top # Muestra los procesos ejecutándose en cada contenedor

docker compose ps # lista los contenedores creados por docker compose

Notas

Notas – Evitando el sudo

- Para ejecutar Docker con un usuario sin privilegios:

  sudo usermod –aG docker $USER #añadimos el usuario al grupo de usuarios Docker

  newgrp docker #para aplicar los cambios sin reiniciar sesión

Notas – Limitando los recursos del contenedor

- Verificar los recursos del Docker host:

  nproc –all # número de procesadores

  free –h # memoria ram

- Asignar recursos al contenedor

  docker run -d --memory 512m --cpus 0.5 --name servidor_web httpd:2.4

  ó cambiándolos en tiempo de ejecución

  docker update --memory 1g --cpus 1.5 servidor_web